# Design and implementation of an encrypted instruction set processor on FPGA

George David Tsitlauri and Georgios Dimitriou
Department of Informatics and Telecommunications
University of Thessaly, Lamia, Greece
gdtsitlauri@gmail.com

***Abstract*—CryptoCPU executes AES-128-encrypted programs with a six-stage, in-order MIPS32-subset processor. Address-dependent AES-XEX protects instruction blocks; an internal cache reuses decrypted instructions without writing them to memory. Code, tweak and data-operation keys are separate inputs. The C++ implementation passes AES known-answer checks, while fourteen directed programs and 5,000 randomized runs agree with a reference interpreter. Three injected pipeline faults are detected. At the eight-line cache design point, encryption increases summed modeled cycles across ten benchmarks by 1.5%, although individual overhead reaches 133.8% for a short program. Ciphertext experiments characterize execution failures without asserting authentication. Resource estimates are given for an Artix-7 FPGA.**

***Index Terms*—encrypted instructions, AES-128, XEX, processor pipeline, FPGA, high-level synthesis**

## I. Introduction

CryptoCPU studies encrypted execution with a decryption stage between fetch and decode. Its central idea is to retain ciphertext in instruction storage, decrypt blocks inside the core, and cache recovered instructions so that loops amortize the cryptographic work. The design comprises a six-stage pipeline, separate instruction and data memories and an assembly/encryption toolchain. Standard subset encodings, separate keys and reference state comparison keep the implementation precise and verifiable.

AES supplies the block cipher [1]. ECB preserves equality between repeated plaintext blocks [2], including instruction padding. Tweakable encryption provides context-dependent transforms [3]; CryptoCPU uses XEX with an AES-derived tweak for each nonce and block address. The contribution is the executable architecture and reproducible evaluation of its behavior and modeled cost, not a new cipher or authentication scheme. The repository contains the source, tests and verification manifests.

## II. Architecture and Protection Boundary

### A. Pipeline and Memory Organization

The stages are instruction fetch (IF), decrypt/cache access (DC), instruction decode (ID), execute (EX), memory access (MEM) and write-back (WB). Fig. 1 shows their relationship. Forwarding resolves supported data dependencies, a load-use detector inserts a stall, and branches resolve in EX with younger instructions squashed. The ISA uses no branch delay slots. Faults stop execution at retirement after older instructions complete; the output state includes the termination status and end PC.

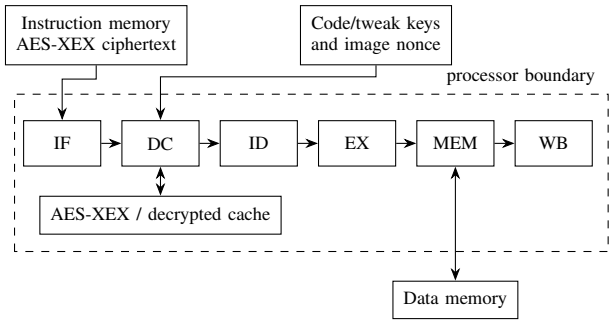



Fig. 1. Logical instruction path. Plaintext instruction blocks remain inside the core. AES data instructions operate through MEM using a separate data key.

The supported MIPS32 subset includes arithmetic, logic, shifts, multiply/divide with HI/LO, byte/halfword/word loads and stores, branches, jumps and calls. Instruction storage contains 4,096 words at base address `0x00400000`; data storage contains 4,096 words at `0x10010000`. Each is 16 KiB. The stack starts at the end of data memory. These addresses follow the MARS convention, without implying complete MIPS compatibility.

Four instructions form a 128-bit AES block. The direct-mapped decrypted cache has eight active lines by default: 128 bytes of instruction data, excluding tags. Supported configurations use powers of two from one to 64 lines. The C++ arrays reserve the maximum capacity, so an active-line setting alone does not determine synthesized storage. The core exposes instruction memory as a read-only input and has no write path that copies decrypted instructions into data memory. Plaintext appears in its cache, pipeline and internal temporaries.

### B. Instruction Encryption and Separate Keys

Let $P_b$ and $C_b$ denote plaintext and ciphertext at block index $b$. The repository image format uses

$$T_b = E_{K_t}(N_0 \,\|\, N_1 \,\|\, 0 \,\|\, b), \tag{1}$$

$$C_b = E_{K_c}(P_b \oplus T_b) \oplus T_b, \tag{2}$$

$$P_b = D_{K_c}(C_b \oplus T_b) \oplus T_b. \tag{3}$$

Here $E$ and $D$ are AES-128 encryption and decryption, $K_c$ is the code key, and $K_t$ is the tweak key. Each concatenated

field is 32 bits; $N_0, N_1$ form a public 64-bit image nonce. Words enter AES in most-significant-byte-first order. This is a defined AES-XEX block format, not an XTS storage format. Encryption remains deterministic at a fixed key, nonce and block index; different images should use distinct nonces when the key pair is reused.

A third key, $K_d$, serves the private `aesenc` and `aesdec` instructions (opcodes `0x3A` and `0x3B`). They transform a 16-byte block in data memory, using source and destination addresses held in registers. Separate data and code keys prevent this interface from directly decrypting instruction ciphertext. Ordinary data memory is otherwise unencrypted.

### C. Threat Model

The adversary may inspect or modify instruction ciphertext, but cannot access internal plaintext or code/tweak keys. Hardware provisioning, access control and debug restrictions enforce that boundary; a C++ key argument alone cannot. XEX has no authentication tag, and corrupted plaintext may decode into valid instructions. Traps therefore do not establish integrity. Replay, access patterns, compromised provisioning and physical side channels are outside the evaluated protection.

## III. Implementation and Evaluation Method

### A. Software and Reference Execution

The C++17 design uses fixed-size arrays and HLS top function `cryptocpu_top`. The toolchain includes an assembler, image tool, sequential reference interpreter and self-checking testbenches. The interpreter independently implements decode/execute but shares the AES routines; known-answer tests provide the separate cryptographic check. Data-label alignment and strict key-file/argument handling have targeted regressions.

The directed testbench compares status, end PC, retired count, all 32 general registers, HI/LO and complete data memory; fixed expected values additionally check intended program behavior. Source/binary hashes, commands, versions, exit codes and actual verification timestamps are recorded in `results/host/manifest.json`. The recorded run uses Python 3.13.16 and LLVM-MinGW/Clang 23.1.1 on Windows.

### B. Cycle Accounting

One main-loop iteration is one modeled pipeline cycle. At the design point, an instruction-block miss costs a four-cycle memory delay plus an eleven-cycle AES service delay when encryption is enabled. A cache hit reuses the decrypted block without this service. The plaintext baseline retains the same pipeline, cache and memory delay. Data AES instructions use the configured AES latency.

These parameters abstract implementation timing. In particular, the model does not separately schedule tweak derivation and block decryption; setup and key expansion precede cycle accounting, and multiply/divide execute in one modeled cycle. An HLS schedule need not map a model iteration to one FPGA clock cycle. Performance is reported as cycles and cycles per retired instruction (CPI), without converting it into a physical frequency or throughput.

TABLE I
Host verification results

| Check | Observed result |
|---|---|
| AES known-answer vectors | All pass, both directions |
| XEX randomized round trips | 10,000 pass |
| Directed programs | 14/14 pass |
| Random differential runs | 5,000; zero mismatches |
| Tooling / classification regressions | 6 methods / 8 cases pass |
| Forwarding mutant | Detected in 500/500 runs |
| Load-use mutant | Detected in 405/500 runs |
| Branch-squash mutant | Detected in 495/500 runs |

### C. Test Campaigns

AES encryption and decryption are checked against FIPS-197 examples and four SP 800-38A AES-128 blocks [1], [2]; 10,000 randomized XEX round trips check inversion. Fourteen directed programs exercise computation, dependencies, control flow, AES operations, data alignment and three exception cases. Random differential testing generates 1,000 programs using seed 2026 and runs each in five configurations. Mutation testing deliberately removes forwarding, a load-use stall or a branch squash, with 100 programs in five configurations per mutant (seed 7).

For ciphertext experiments, ten programs are executed with a wrong code key, 500 single-bit flips each (seed 1), and pairs of swapped code blocks. Classification distinguishes traps, timeouts, normally halted changed states and normally halted unchanged states. Comparison includes PC and HI/LO as well as registers and memory. The timeout budget is $20C + 1000$ modeled cycles for baseline cost $C$. The bit-flip sequence uses the C runtime's `rand()`, so exact samples depend on the recorded runtime as well as the seed.

## IV. Results and Discussion

### A. Functional Checks

Table I summarizes the host results. All directed programs pass. Random execution produces 4,455 normal halts, 430 memory traps and 115 overflow traps, with zero mismatches against the reference. The targeted tooling tests include multiple invalid-input subcases; the outcome-classifier tests include programs differing only in HI, LO or end PC.

The three detected mutants establish sensitivity to those defects, not exhaustive correctness.

### B. Ciphertext Observations

Across the ten benchmark images, XEX produces 1,024 distinct ciphertext blocks per 16-KiB image. ECB preserves the 4–11 distinct plaintext patterns present in each image, including padding. The directed-test memory scan finds no complete, nonzero 16-byte instruction blocks in the ciphertext image or final data memory. This check does not address fragments, transient leakage or physical observation.

TABLE II
OBSERVED CIPHERTEXT-MODIFICATION OUTCOMES

| Experiment | Runs | Traps | Timeouts | Changed halt |
|---|---|---|---|---|
| Wrong code key | 10 | 10 | 0 | 0 |
| Single-bit flip | 5,000 | 4,995 | 5 | 0 |
| Block swap | 208 | 208 | 0 | 0 |

TABLE III
CYCLE-MODEL PERFORMANCE AT THE DESIGN POINT

| Program | Retired | Plain cycles | XEX cycles | Increase |
|---|---|---|---|---|
| Signature | 16 | 37 | 81 | 118.9% |
| Loop sum | 408 | 722 | 755 | 4.6% |
| Fibonacci | 19,731 | 36,532 | 36,609 | 0.2% |
| Bubble sort | 957 | 1,562 | 1,628 | 4.2% |
| Strings | 171 | 311 | 377 | 21.2% |
| Mul/div | 33 | 74 | 173 | 133.8% |
| AES data | 11 | 45 | 68 | 51.1% |
| Hazards | 37 | 91 | 179 | 96.7% |
| Sieve | 3,174 | 5,250 | 5,349 | 1.9% |
| Matrix multiply | 1,146 | 1,444 | 1,554 | 7.6% |
| Total | 25,684 | 46,068 | 46,773 | 1.5% |

Table II records 99.9% traps and 0.1% timeouts for bit flips; no modified run halts with a changed or unchanged final state. These sampled outcomes do not establish arbitrary-tamper detection for an unauthenticated format.

### C. Modeled Performance

Table III compares the same workloads with and without instruction encryption. All use a cold cache, eight active lines, four-cycle memory reads and eleven-cycle AES service. The encrypted total is 46,773 cycles versus 46,068 plaintext cycles: an increase of 705 cycles, or 1.5%. This ratio is computed from summed cycles, not from an unweighted mean of per-program overheads; Fibonacci dominates the aggregate execution time.

The five loop-oriented benchmarks incur 0.2–7.6% overhead. Short programs pay proportionally more for initial misses: the 33-instruction multiply/divide benchmark rises from 74 to 173 cycles. The cache-size sweep exposes the same tradeoff. With one active line, the loop-oriented workloads have CPI values of 5.911–9.130; with eight lines, they reach 1.356–1.855. Increasing capacity past eight lines does not improve these small benchmark results.

The latency sweep varies modeled AES service from one to 44 cycles. Fibonacci changes only from 1.852 to 1.867 CPI at eight lines, whereas the short multiply/divide program changes from 2.515 to 14.242 CPI. Consequently, the measured model advantage comes from locality and reuse; it does not predict the overhead of arbitrary applications or a specific AES hardware design.

## V. FPGA IMPLEMENTATION

### A. Thesis Prototype

The thesis prototype [4] followed the complete Vitis HLS and Vivado flow [5], [6]. C simulation executed the encrypted demonstration program and passed its signature check; C synthesis produced the RTL, and C/RTL co-simulation reproduced the C-simulation signature, register dump and memory dump. The core was packaged as an IP block and placed in a Vivado block design with a clocking wizard, a processor-system reset and a constant `ap_start`. Its `ap_done` output drives an LED.

The design was implemented on an AMD Artix-7 XC7A200T (AC701 evaluation board), with the 200 MHz differential system clock on pins R3/P3 and the LED on pin M26. Timing closed with positive slack, and the bitstream ran the encrypted program on the board, where the LED confirmed completion. Table IV lists the implementation results. That prototype decrypted instruction blocks with AES-128 in ECB mode.

### B. Estimated Cost of CryptoCPU

The estimates in Table IV start from the prototype and add the differences in the datapath. The prototype's AES core already encrypts and decrypts, so XEX adds mainly a 128-bit tweak register, two 128-bit XOR stages and the multiplexing that lets the core derive the tweak, well under 1% of the XC7A200T's 134,600 LUTs. The wider subset adds a 32-bit divider, HI/LO and byte/halfword alignment, about 2–3% more; the multiplier, as in the prototype, maps to DSP slices. The prototype kept code and data in one 1 KiB memory with a four-line instruction cache; CryptoCPU's 16 KiB instruction and 16 KiB data memories need about eight further 36-Kbit block RAMs (2% of 365), and the eight-line decrypted cache adds a few hundred flip-flops. The clock target follows the prototype, with the divider scheduled over several cycles so that the AES round remains the critical path. These figures are estimates, not reports of an implementation of CryptoCPU.

## VI. CONCLUSION

CryptoCPU realizes encrypted execution with separate keys, address-dependent encryption, standard subset decoding and verification against a reference interpreter. The evaluated programs agree with the reference, while a small decrypted cache amortizes modeled AES cost through reuse. Authentication and physical protection remain distinct concerns.

Crypto3DStackCPU [7] continues this direction through a four-tier memory model, encrypted and authenticated program images, and protected transfers between tiers. It has its own implementation and validation; CryptoCPU provides the encrypted-instruction foundation.

TABLE IV
ARTIX-7 XC7A200T IMPLEMENTATION

| Quantity | Thesis prototype (measured) | CryptoCPU (estimated) |
|---|---|---|
| Clock | 75 MHz | 75 MHz |
| LUTs | ≈15% | 17–19% |
| Flip-flops | ≈10% | 10–11% |
| Block RAM | ≈12% | 13–15% |
| DSP slices | ≈5% | ≈5% |
| Power (Vivado estimate) | ≈0.6 W | 0.6–0.7 W |

## REFERENCES


[1] National Institute of Standards and Technology, "Advanced encryption standard (AES)," U.S. Department of Commerce, Tech. Rep. FIPS 197, 2001, editorial update, 2023. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final

[2] M. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," National Institute of Standards and Technology, Tech. Rep. SP 800-38A, 2001. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/38/a/final

[3] P. Rogaway, "Efficient instantiations of tweakable blockciphers and refinements to modes OCB and PMAC," in *Advances in Cryptology—ASIACRYPT 2004*, ser. Lecture Notes in Computer Science, vol. 3329. Springer, 2004, pp. 16–31. [Online]. Available: https://web.cs.ucdavis.edu/~rogaway/papers/offsets.html

[4] G. D. Tsitlauri, "Design and implementation of an encrypted instruction set processor on FPGA," B.Sc. thesis, University of Thessaly, Department of Informatics and Telecommunications, 2025, in Greek. Supervisor: Assistant Prof. Georgios Dimitriou.

[5] AMD, *Vitis High-Level Synthesis User Guide (UG1399): Automatically Verifying the RTL*. [Online]. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls/Automatically-Verifying-the-RTL

[6] ——, *Vivado Design Suite User Guide: Implementation (UG904), Viewing Implementation Reports*. [Online]. Available: https://docs.amd.com/r/en-US/ug904-vivado-implementation/Viewing-Implementation-Reports

[7] G. D. Tsitlauri, "Crypto3DStackCPU," Source repository. [Online]. Available: https://github.com/gdtsitlauri/Crypto3DStackCPU